

Agent Trajectory Logger + Intent Drift Detector

1. Overview

This document outlines a design for a lightweight observability layer for LLM agents: one that logs step-by-step trajectories and surfaces a real-time signal when an agent's behavior starts to diverge from its declared intent. The project brief provides a solid starting point. This document builds on it with some architectural suggestions, a reframing of the core detection problem, and a few honest questions about the system's assumptions that are worth working through before locking in direction.

2. A Different Way to Think About Drift

The brief proposes computing cosine similarity between the declared intent and each agent step individually. That is a reasonable baseline, and it works well for simple linear tasks. For more complex agent runs though, per-step similarity tends to produce a specific kind of noise: a mid-trajectory retrieval step will look semantically distant from the top-level intent even when the agent is doing exactly what it should. Flagging that as drift creates false positives that make the signal hard to trust.

A few ideas worth exploring together:

- Trajectory-level coherence as the primary metric. Rather than scoring each step in isolation, embed the cumulative step sequence as a running context and measure coherence against the declared intent at checkpoint intervals. Per-step scores remain available as a diagnostic layer, but the primary alert signal comes from trajectory-level divergence.
- Distinguishing unrecovered drift from recovered drift. An agent that drifts and then self-corrects is behaving adaptively. An agent that drifts and keeps drifting is the real failure mode. Tracking recovery events separately makes the signal much more actionable.
- A slightly richer intent schema. A single sentence works fine for short tasks. For longer runs it might be worth supporting optional constraints and sub-intent milestones. A simple JSON structure at run initialization would be enough to start.

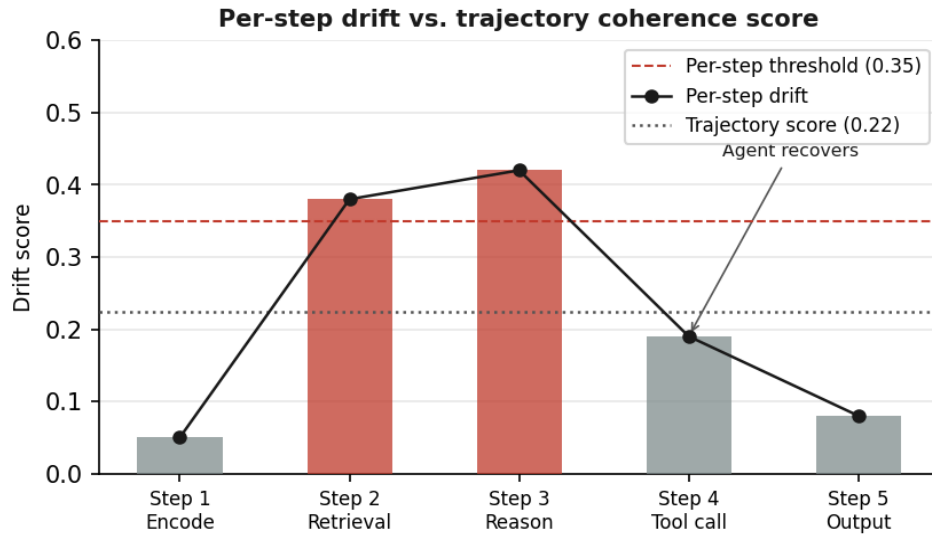


Figure 1. An example 5-step agent run. Steps 2 and 3 cross the per-step alert threshold but the agent recovers by Step 4. The trajectory coherence score (dotted line) stays below the threshold throughout, telling a more stable and accurate story.

3. Proposed Architecture

The brief suggests OpenAI embeddings, a LangChain wrapper, and SQLite or JSONL for storage. Those are all reasonable choices to get started. The stack below swaps in NVIDIA-native tooling that reflects how production agentic systems are typically instrumented, and that makes the system deployable in enterprise environments without modification.

Component	Brief's suggestion	Proposed stack
Agent runtime	LangChain wrapper	NeMo Agent Toolkit
Trajectory storage	SQLite / JSONL	OpenTelemetry + Phoenix (local)
Embeddings	OpenAI API	NIM: nv-embedqa-e5-v5
Similarity search	FAISS	FAISS / CuVS <i>FAISS for the prototype. CuVS (NVIDIA GPU-accelerated vector search) is the natural upgrade at scale, particularly relevant if this becomes a product with many concurrent runs or long agent histories.</i>
Dashboard	Streamlit	Streamlit, then React in iter 2
Cost overlay	Token count	NIM token + compute cost metrics

Table 1. Stack comparison.

System Architecture

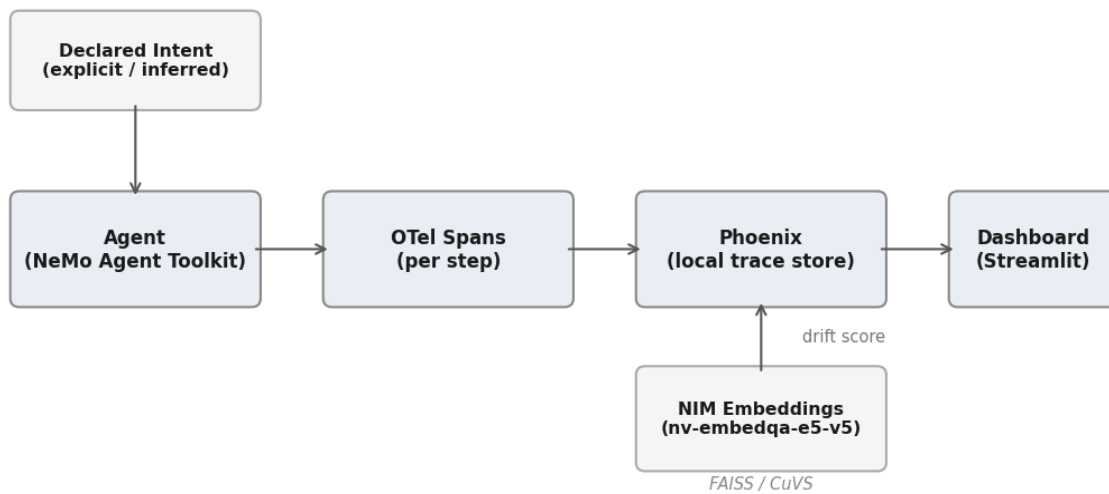


Figure 2. System architecture overview.

Trajectory Logger: NeMo Agent Toolkit + OpenTelemetry + Phoenix

NeMo Agent Toolkit exposes native hooks at every tool call boundary, so the wrapper layer is thin. Each step emits an OpenTelemetry span with tool name, inputs, outputs, latency, and token count. Phoenix (Arize) runs locally as the trace store and provides a query interface over those spans with no cloud infrastructure needed. This replaces hand-rolled JSONL with a pattern that is standard in production ML observability, which matters if the system is ever adopted by teams that already have OTel instrumentation in place.

Intent Drift Detector: NIM Embeddings + FAISS / CuVS

Swapping OpenAI embeddings for a NIM microservice (nvidia/nv-embedqa-e5-v5) keeps the system deployable in air-gapped enterprise environments where sending data to an external API is not an option. FAISS handles similarity search for the prototype and is the right call to start. CuVS, NVIDIA's GPU-accelerated vector search library, becomes the natural upgrade if the system scales to many concurrent runs or long agent histories where lookup latency starts to matter. The detection logic follows the trajectory-level framing above.

On Declared Intent: Where Does It Come From?

The brief treats declared intent as a string passed at run start, which is clean to implement but quietly assumes that intent is easy to obtain and reliable. In practice that assumption deserves some scrutiny. There are a few realistic paths to sourcing intent, each with different tradeoffs:

- Explicit user input. The user types an intent string before the run. Clean and unambiguous, but puts burden on the user. People often do not know how to articulate intent precisely enough for

a detector to be useful.

- Inferred from the prompt. Intent is extracted automatically from the initial user message using a lightweight LLM call. More realistic for real deployments, but introduces a noisy extraction layer upstream of the detector. Errors there compound into the drift signal.
- Structured task definition. The agent is initialized with a formal task spec, which becomes the intent. This is how many production agentic systems already work and gives the cleanest signal, but assumes the upstream system is well-designed.
- Agent-generated plan. The agent produces a step-by-step plan at Step 0, and that plan becomes the intent schema it is measured against. Interesting because it does not require any external input, but a poorly planned agent effectively sets its own low bar.

There is also a temporal dimension worth flagging: intent can evolve mid-run. A user who starts with 'find me a hotel in Paris' might refine to 'actually, near the Marais' after seeing early results. A static declared intent cannot handle that gracefully. Dynamic intent tracking is probably out of scope for the prototype, but it is a known limitation worth naming explicitly in the system's documentation.

The choice of intent sourcing strategy is an early alignment decision, since it shapes both the user experience and the reliability of everything downstream.

Dashboard: Streamlit, then React

Streamlit is a good choice for the first iteration. The one addition worth making early is a cost overlay: NIM token and compute cost per step alongside the drift score. That turns misalignment cost into a number a founder or enterprise customer can act on. The run comparison view (same intent, different agent configs side by side) is probably the highest value feature for the first demo.

Fallback Path

NeMo Agent Toolkit has a steeper setup curve than LangChain. If Week 1 integration takes longer than expected, the practical fallback is LangChain + OTel + Phoenix + NIM embeddings. That keeps the most differentiating parts of the stack while reducing early risk.

4. Implementation Plan

Week	Focus	Deliverable
1	Core loop	NeMo agent wrapper, OTEL spans, Phoenix local store, NIM embeddings, drift score working end to end.
2	Dashboard + demo	Streamlit UI with trajectory timeline and drift visualization. Tested on document QA, code gen, and data retrieval.
3	Iteration + handoff	Threshold tuning, edge case testing, pip-installable SDK packaging, deployment notes for NVIDIA AI Enterprise.

Table 2. Three-week plan.

5. On Calibration and Ground Truth

The architecture section describes how to detect drift. What it does not address is how you would know whether the detector is working. This is worth thinking through early because it affects design decisions in Week 1, not just Week 3.

The core question is: what is ground truth here? If the threshold is set at 0.35, how do you know that is the right number? The most honest answer is that drift detection is a proxy metric, and like all proxy metrics it needs calibration against something real. The natural starting point is human judgment: take a set of agent runs, have people label which ones went wrong, and see whether the drift scores correlate. That labeled set becomes the eval dataset.

But there is a deeper issue underneath that. Drift from declared intent does not always equal task failure. An agent could diverge significantly from its stated intent and still produce a correct output through a different path. So the question becomes whether this system is optimizing for intent fidelity (did the agent do what you asked?) or outcome quality (did the agent produce a good result?). Those are related but not the same thing, and the system's value proposition depends on which one matters more in the target use case.

A few practical questions that follow from this:

- Should the system expose a feedback mechanism so users can flag false positives and false negatives? That data would be invaluable for threshold calibration over time.
- Is there a way to validate drift scores against task outcomes automatically? For document QA, the final output can be evaluated for correctness independently, which gives a weak supervision signal without requiring human labeling of every run.
- What does a useful drift score look like to the person reading the dashboard? A number between 0 and 1 is technically correct but may not be intuitive. It might be worth thinking about how to present the signal in terms of risk level or confidence rather than raw cosine distance.

6. A Thought on Packaging

One thing worth discussing: there is a natural product framing here that goes beyond a middleware layer. NeMo Guardrails intercepts agent behavior prospectively, defining rails before a bad output happens. This system is the retrospective complement: it observes what actually happened and surfaces when the agent diverged from intent. These are two sides of the same problem, and together they form a more complete observability story.

A clean pip-installable package with a simple API surface would position this as a product rather than a script. Something like:

```
from driftguard import DriftGuard

dg = DriftGuard(
    embedding_backend="nim",
    storage_backend="phoenix",
    drift_mode="trajectory",
)

with dg.run(intent="summarize contract and flag risks") as session:
    result = agent.invoke(task)
    report = session.report()
    # drift_score, recovery_events, cost_overlay, flagged_steps
```

The brief already gestures at SDK packaging in Week 3. The question worth aligning on early is whether the package is the point of the project rather than an afterthought: a standalone open-source observability tool for agentic AI, or the observability layer inside a larger product. Those lead to meaningfully different design decisions around the API surface, extensibility, and what gets prioritized in Week 1.

7. Questions for Review and Discussion

A few things worth aligning on before locking in direction:

- What agent scenarios matter most? The drift detector behaves differently on document QA (long, structured), code gen (high tool diversity), and data retrieval (short, repetitive). Threshold calibration and the intent schema design both depend on this.
- How should users express intent? The choice between explicit input, prompt inference, structured task spec, or agent-generated plan has downstream consequences for detection reliability and user experience. Worth deciding early.
- Is the primary user internal (tooling for your own agents) or external (a product other teams use to observe theirs)? This shapes the SDK design and what the dashboard needs to do.

- Intent fidelity or outcome quality? Understanding which of these the system is primarily optimizing for will help calibrate what ground truth to validate against, and how to present the drift signal to users.
- Standalone package or embedded layer? The packaging question is a product strategy question as much as a technical one. Happy to think through both directions.